

Project Instructions:

Phase 1 Lab & Hashing Setup :

Download Linux:

<https://ubuntu.com/download/desktop>

Open VMware Workstation

Launch **VMware Workstation Player**

Click: Create a New Virtual Machine

Choose Installation Method

Select:

Installer disc image file (ISO)

Click **Browse**

Select the Linux ISO you downloaded.

Example:

ubuntu-22.04-desktop.iso

Set Disk Size

Recommended: 40GB

Memory (RAM)

If you have **16 GB RAM**

8 GB – minimum

Network

Use:

NAT

Install Linux

Select Language

Click Install Ubuntu

Choose Normal Installation

Select Erase disk and install Ubuntu

(This only erases the virtual disk, not your real computer)

Install VMware Tools (Very Important):

Open Ubuntu terminal and type:

```
sudo apt update
```

```
sudo apt install open-vm-tools-desktop
```

```
sudo reboot
```

Update system:

```
sudo apt update
```

```
sudo apt upgrade
```

Install Hashcat / John the Ripper:

```
sudo apt install john
```

Type : john

To confirm the version

```
sudo apt install hashcat
```

```
Verify: hashcat --version
```

Step 4 — Install SQLite Database

```
sudo apt install sqlite3
```

Verify: sqlite3 --version

Create Project Folder:

```
mkdir password_cracking_lab
```

```
cd password_cracking_lab
```

Create SQLite Test Database:

```
sqlite3 users.db
```

```
CREATE TABLE users (  
id INTEGER PRIMARY KEY,  
username TEXT,  
password TEXT  
);
```

Insert **weak test passwords**:

```
INSERT INTO users (username,password) VALUES ('user1','pass123');
```

```
INSERT INTO users (username,password) VALUES ('user2','123456');
```

```
INSERT INTO users (username,password) VALUES ('user3','qwerty');
```

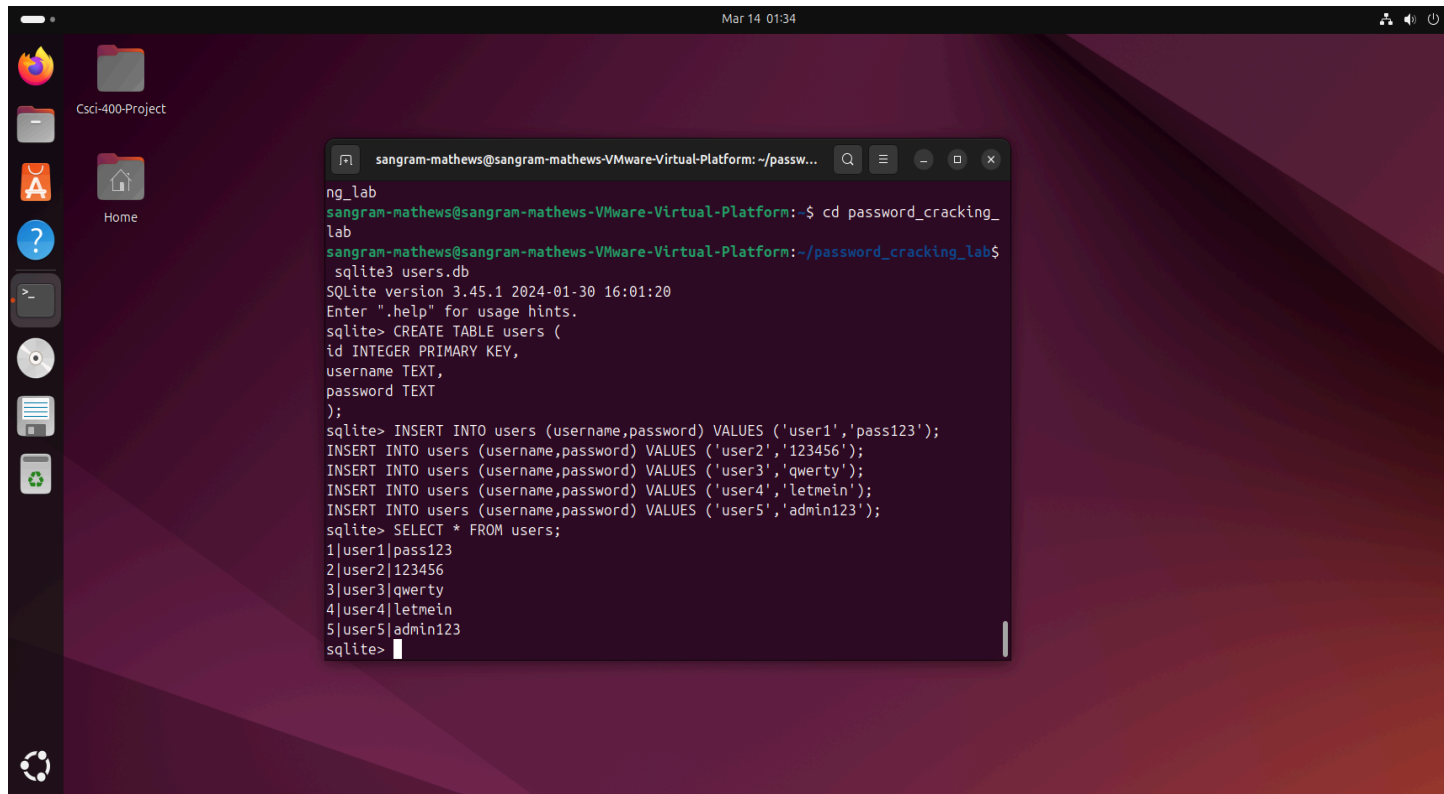
```
INSERT INTO users (username,password) VALUES ('user4','letmein');
```

```
INSERT INTO users (username,password) VALUES ('user5','admin123');
```

View the data:

```
SELECT * FROM users;
```

```
.exit
```



Generate SHA-256 Hashes:

nano hash_passwords.py

Paste this Code:

```
import sqlite3
```

```
import hashlib
```

```
conn = sqlite3.connect("users.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute("SELECT username,password FROM users")
```

```
for row in cursor.fetchall():
```

```
    username = row[0]
```

```
password = row[1]
```

```
hashed = hashlib.sha256(password.encode()).hexdigest()
```

```
print(username + ":" + hashed)
```

Save and exit

Run:

```
python3 hash_passwords.py > hashes.txt
```

Now you created:

hashes.txt

Create Dictionary File:

```
nano wordlist.txt
```

Example:

123456

password

pass123

admin

admin123

qwerty

letmein

welcome

Then Save it

The Command that I put for John the ripper doesn't support SHA-256
So we need to download John Jumbo

First remove the current version:
`sudo apt remove john -y`

Install Required Build Tools:
`sudo apt update`
`sudo apt install git build-essential libssl-dev zlib1g-dev -y`

Download John Jumbo:
`git clone https://github.com/openwall/john.git`

Go into the source folder:
`cd john/src`

Compile John Jumbo:
`./configure`
`make -s clean && make -sj4`

Now go to run folder:
`cd ../run`

Test It:
`./john --list=formats | grep -i sha`

You should now see something like:
Raw-SHA256

Raw-SHA512

HMAC-SHA256

sha256crypt

Run Your Password Crack:

Go back to your main lab folder:
`cd ~/password_cracking_lab`

`pwd`

ls

You should see files like:

users.db

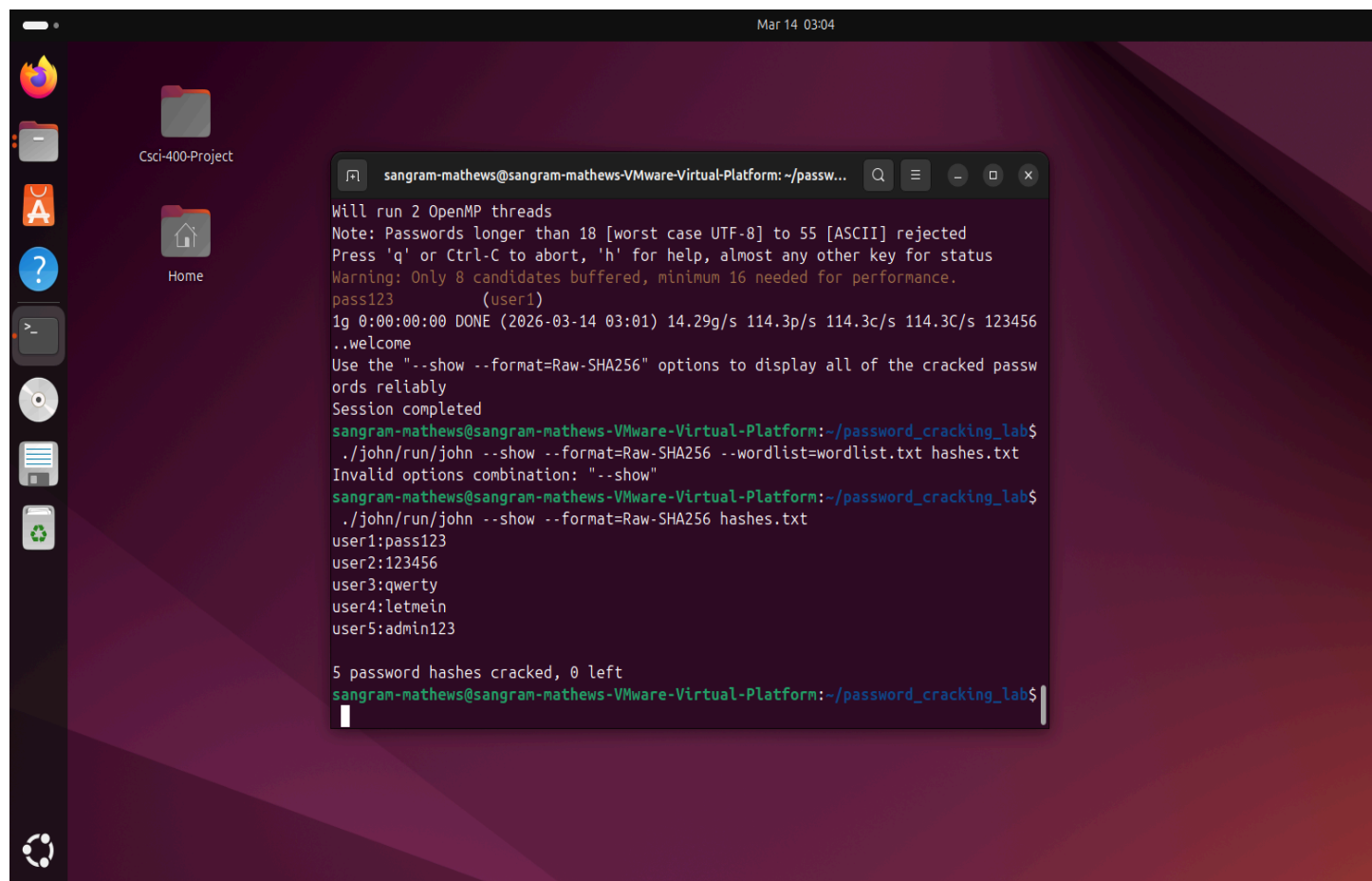
hash_passwords.py

wordlist.txt

john

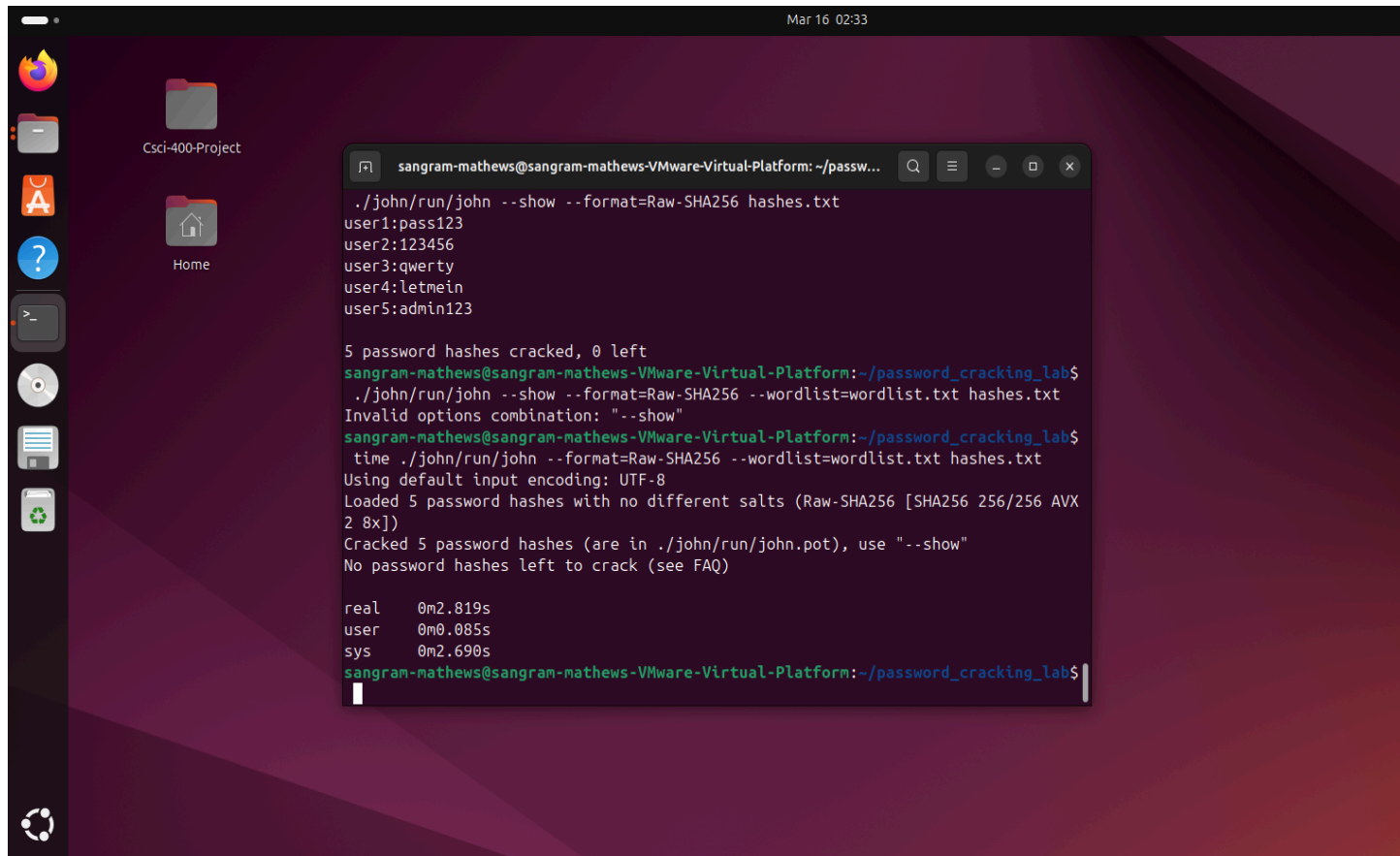
Run John using the correct path:

`./john/run/john --format=Raw-SHA256 --wordlist=wordlist.txt hashes.txt`



Run this command to measure the time:

`time ./john/run/john --format=Raw-SHA256 --wordlist=wordlist.txt hashes.txt`



Phase 2 – Baseline Attack

Dataset creation for 40 Users:

Go to your lab folder:

```
cd ~/password_cracking_lab
```

sqlite3 [users.db](#)

Inside SQLite, paste this:

```
DROP TABLE IF EXISTS users;
```

```
CREATE TABLE users (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    username TEXT NOT NULL UNIQUE,
```



```
password TEXT NOT NULL  
);
```

```
INSERT INTO users (username, password) VALUES
```

```
('user1', '123456'),  
('user2', 'password'),  
('user3', 'qwerty'),  
('user4', 'abc123'),  
('user5', 'letmein'),  
('user6', 'welcome'),  
('user7', 'admin'),  
('user8', 'admin123'),  
('user9', 'pass123'),  
('user10', 'iloveyou'),  
('user11', '123123'),  
('user12', 'monkey'),  
('user13', 'dragon'),  
('user14', 'football'),  
('user15', 'baseball'),  
('user16', 'shadow'),  
('user17', 'master'),  
('user18', 'hello'),  
('user19', 'freedom'),
```

('user20', 'whatever'),
('user21', 'login'),
('user22', 'trustno1'),
('user23', 'secret'),
('user24', 'test123'),
('user25', 'guest'),
('user26', 'superman'),
('user27', 'batman'),
('user28', 'summer'),
('user29', 'winter'),
('user30', 'spring'),
('user31', 'autumn'),
('user32', 'charlie'),
('user33', 'jordan'),
('user34', 'harley'),
('user35', 'michael'),
('user36', 'jessica'),
('user37', 'pepper'),
('user38', 'computer'),
('user39', 'internet'),
('user40', 'grocerygo');

.exit

Check that it worked:

```
sqlite3 users.db "SELECT COUNT(*) FROM users;"
```

Output: 40

Once the database is ready, use your Python script:

```
python3 hash_passwords.py > hashes.txt
```

Then check:

```
cat hashes.txt
```

We are going to use rockyou wordlists from now on:

```
sudo apt install wordlists -y
```

If it doesn't work use this:

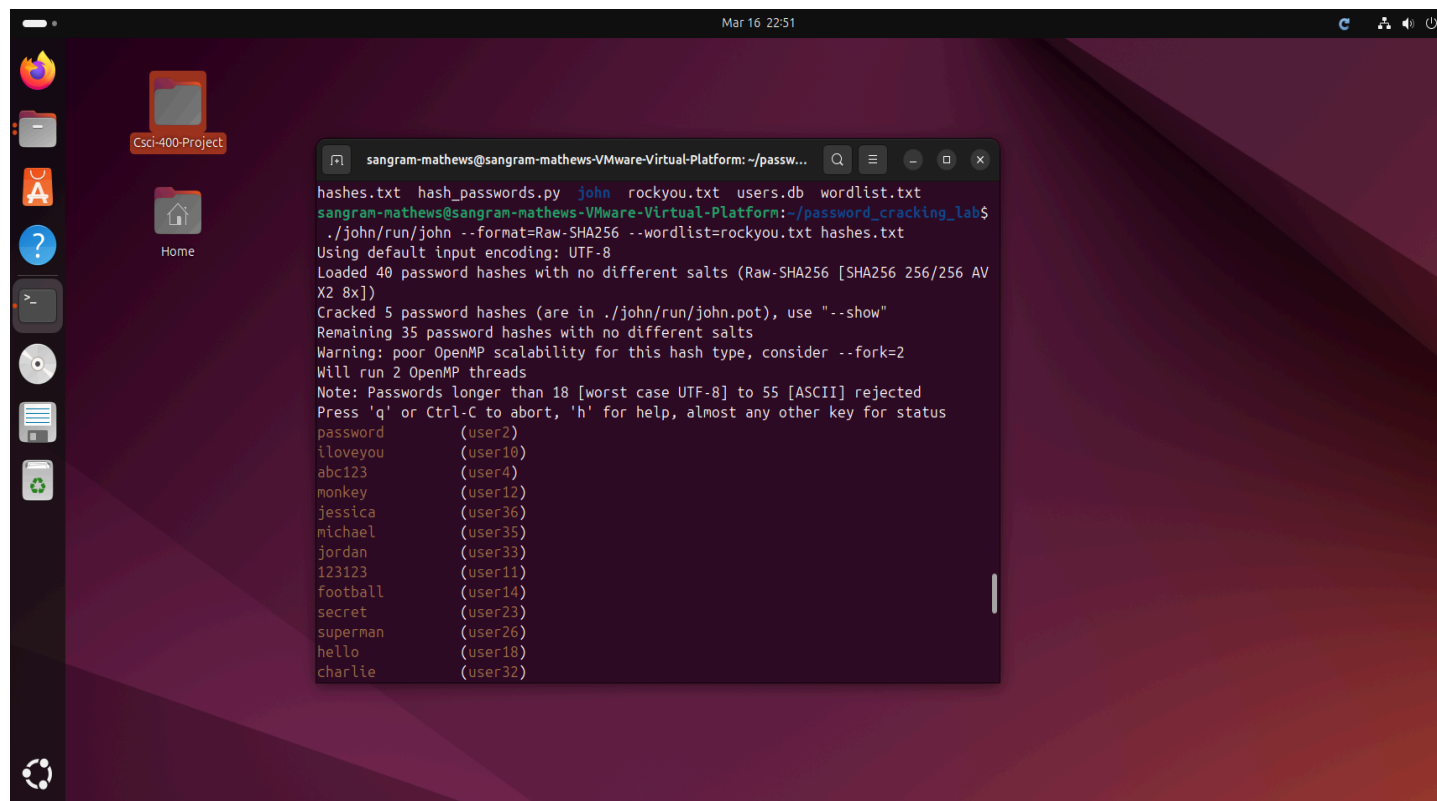
```
wget https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt
```

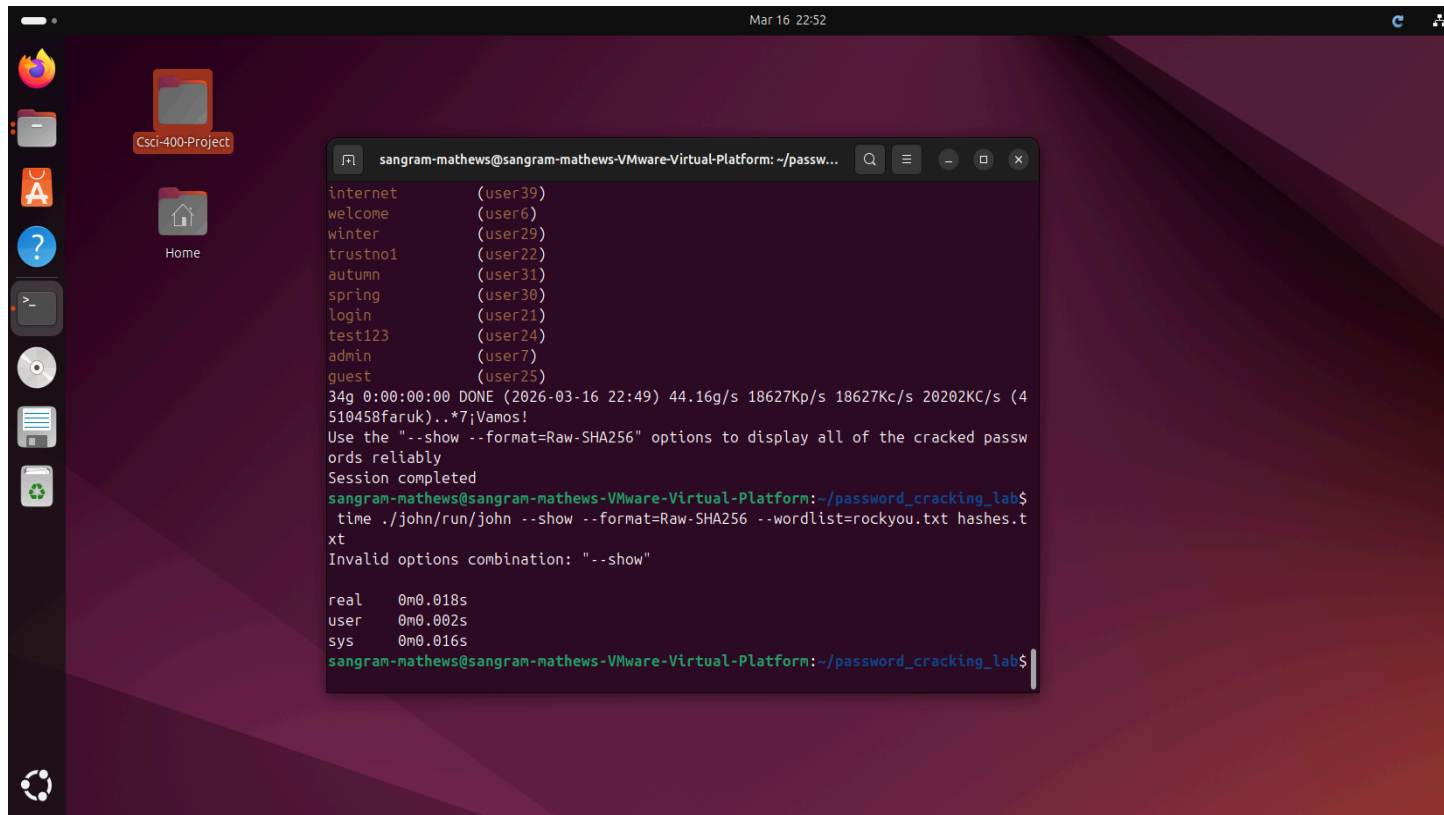
Check if the file exist with:

```
ls
```

Use RockYou with John:

time ./john/run/john --format=Raw-SHA256 --wordlist=rockyou.txt hashes.txt





Phase 3 – Add Salting:

Step 1 — Create New Python Script (Salted Hashing)

Create new script:

```
nano salted_hash_passwords.py
```

Paste this code:

```
import sqlite3
```

```
import hashlib
```

```
import os
```

```
conn = sqlite3.connect("users.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute("SELECT username, password FROM users")
```

```
for row in cursor.fetchall():
```

```
    username = row[0]
```

```
    password = row[1]
```

```
    # Generate random salt
```

```
    salt = os.urandom(16).hex()
```

```
    # Create salted hash
```

```
    salted_password = salt + password
```

```
    hashed = hashlib.sha256(salted_password.encode()).hexdigest()
```

```
    print(username + ":" + salt + ":" + hashed)
```

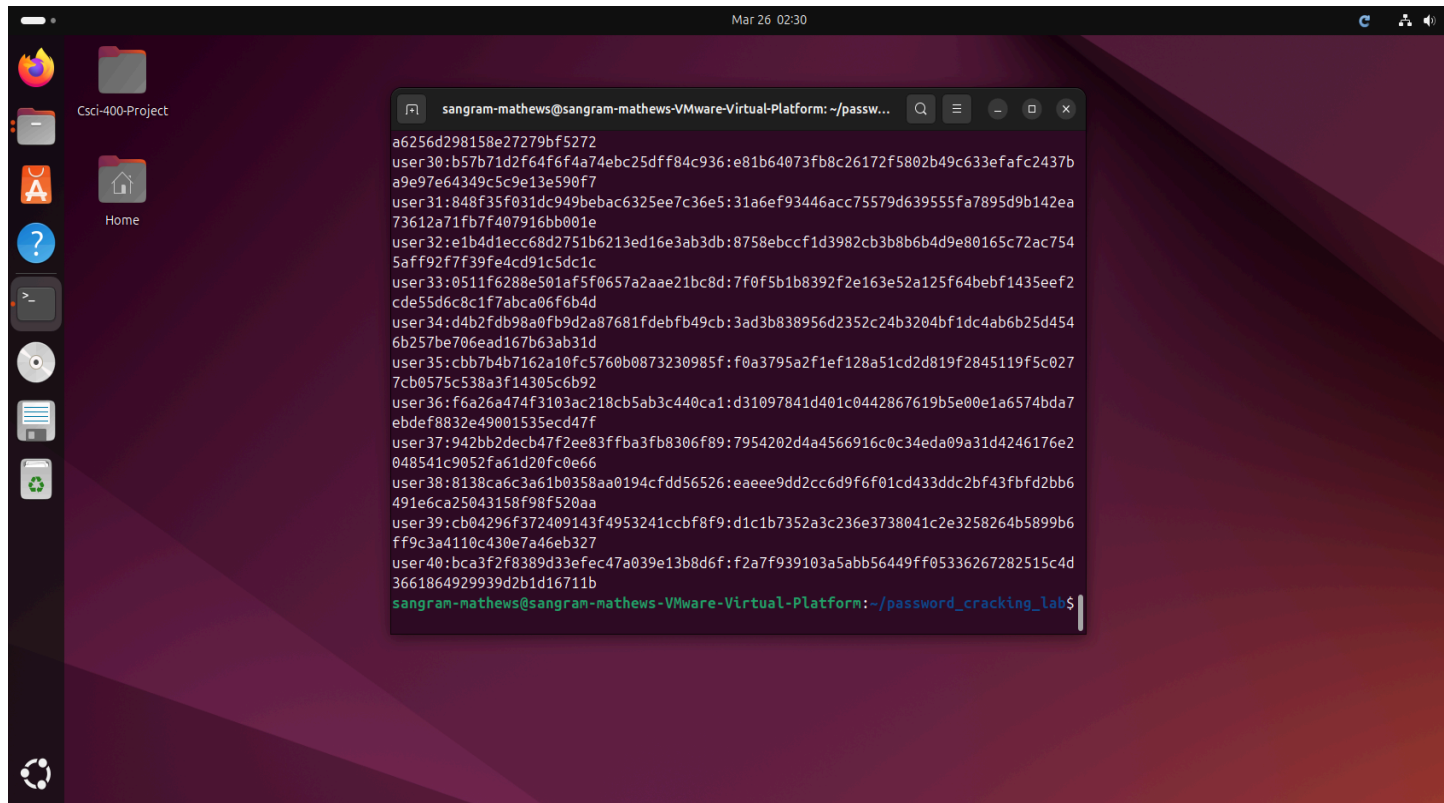
Save and Exit

Step 2 — Generate Salted Hash File

```
python3 salted_hash_passwords.py > salted_hashes.txt
```

Check:

```
cat salted_hashes.txt
```



Step 4 — Try Cracking Salted Hashes

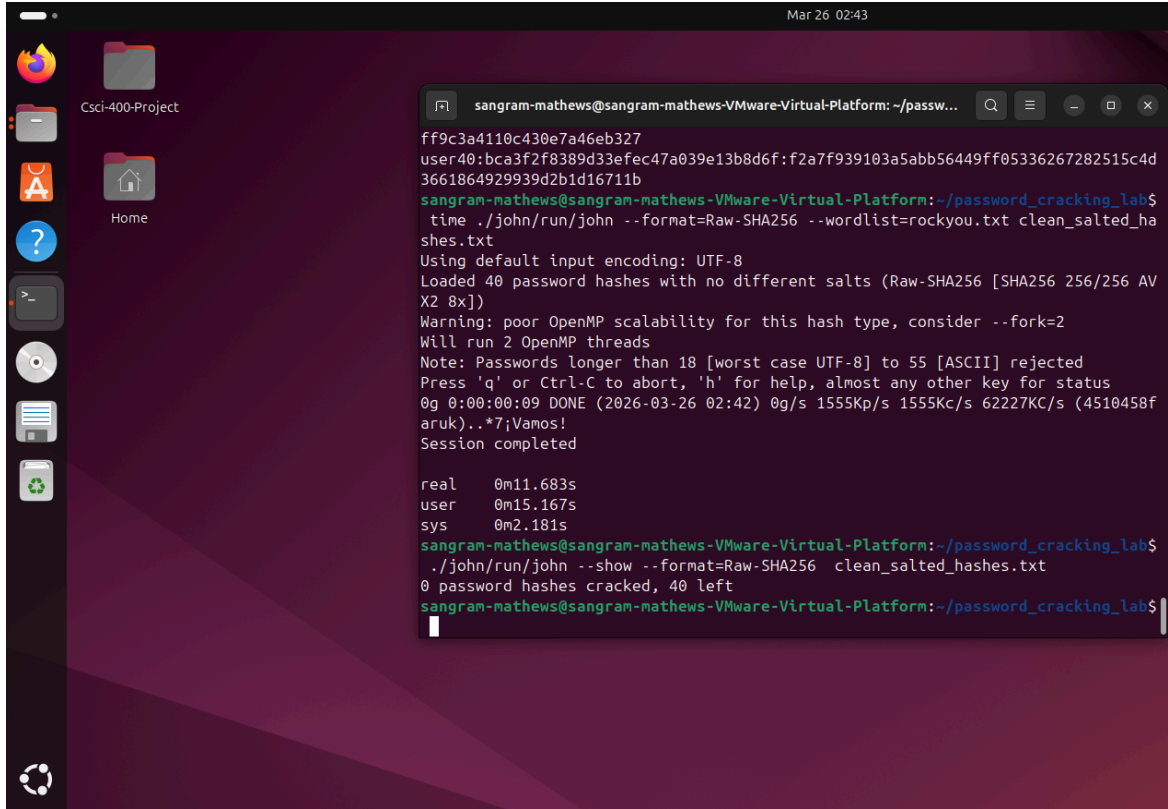
`./john/run/john --format=Raw-SHA256 --wordlist=rockyou.txt salted_hashes.txt`

Step 5 — Show Results

`./john/run/john --show --format=Raw-SHA256 clean_salted_hashes.txt`

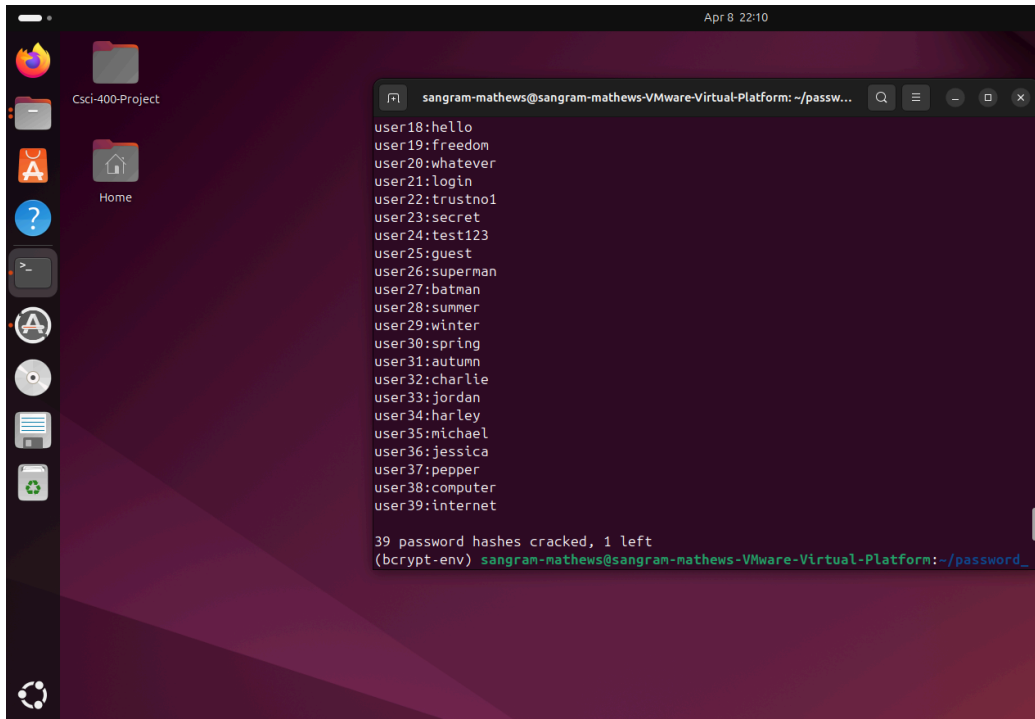
You can compare the time using:

`time ./john/run/john --format=Raw-SHA256 --wordlist=rockyou.txt salted_hashes.txt`



```
sangram-mathews@sangram-mathews-VMware-Virtual-Platform: ~/passw...  
ff9c3a4110c430e7a46eb327  
user40:bca3f2f8389d33efec47a039e13b8d6f:f2a7f939103a5abb56449ff05336267282515c4d  
3661864929939d2b1d16711b  
sangram-mathews@sangram-mathews-VMware-Virtual-Platform:~/password_cracking_lab$  
time ./john/run/john --format=Raw-SHA256 --wordlist=rockyou.txt clean_salted_ha  
shes.txt  
Using default input encoding: UTF-8  
Loaded 40 password hashes with no different salts (Raw-SHA256 [SHA256 256/256 AV  
X2 8x])  
Warning: poor OpenMP scalability for this hash type, consider --fork=2  
Will run 2 OpenMP threads  
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected  
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status  
0g 0:00:00:09 DONE (2026-03-26 02:42) 0g/s 1555Kp/s 1555Kc/s 62227KC/s (4510458f  
aruk)..*7;Vamos!  
Session completed  
  
real    0m11.683s  
user    0m15.167s  
sys     0m2.181s  
sangram-mathews@sangram-mathews-VMware-Virtual-Platform:~/password_cracking_lab$  
./john/run/john --show --format=Raw-SHA256 clean_salted_hashes.txt  
0 password hashes cracked, 40 left  
sangram-mathews@sangram-mathews-VMware-Virtual-Platform:~/password_cracking_lab$
```

Expected Result



```
sangram-mathews@sangram-mathews-VMware-Virtual-Platform: ~/passw...  
user18:hello  
user19:freedom  
user20:whatever  
user21:login  
user22:trustno1  
user23:secret  
user24:test123  
user25:quest  
user26:superman  
user27:batman  
user28:summer  
user29:winter  
user30:spring  
user31:autumn  
user32:charlie  
user33:jordan  
user34:harley  
user35:michael  
user36:jessica  
user37:pepper  
user38:computer  
user39:internet  
  
39 password hashes cracked, 1 left  
(bcrypt-env) sangram-mathews@sangram-mathews-VMware-Virtual-Platform:~/password_
```


39 passwords cracked, much slower than unsalted SHA-256

Your project now demonstrates:

- **SHA256 is weak alone**
- **Salting improves security**
- **Dictionary attacks become harder**

Phase 4 – Implement bcrypt

Step 1 — activate your virtual environment

Since Ubuntu blocked global `pip install`, use the venv you already created:

```
cd ~/password_cracking_lab
```

```
source bcrypt-env/bin/activate
```

You should see `(bcrypt-env)` at the left of the terminal.

Step 2 — install bcrypt inside the venv

```
pip install bcrypt
```

Verify:

```
python3 -c "import bcrypt; print('bcrypt ok')"
```

Step 3 — create the bcrypt hashing script

Create a new file:

```
nano bcrypt_hash_passwords.py
```

Paste this:

```
import sqlite3
```

```
import bcrypt
```

```
conn = sqlite3.connect("users.db")

cursor = conn.cursor()

cursor.execute("SELECT username, password FROM users")

for username, password in cursor.fetchall():

    password_bytes = password.encode("utf-8")

    # cost factor = 12

    hashed = bcrypt.hashpw(password_bytes, bcrypt.gensalt(rounds=12)).decode("utf-8")

    print(f"{username}:{hashed}")
```

Save and exit:

Step 4 — generate bcrypt hashes

Run:

```
python3 bcrypt_hash_passwords.py > bcrypt_hashes.txt
```

Check the file:

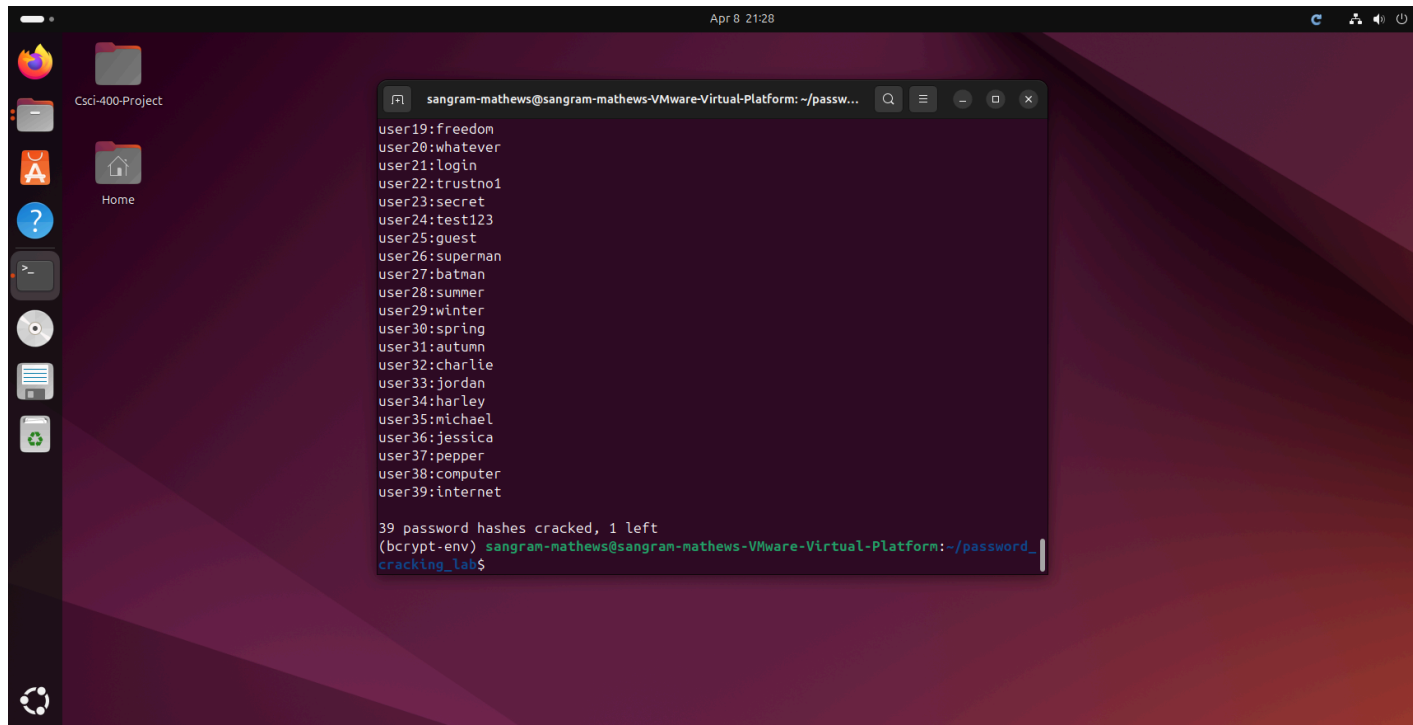
```
head bcrypt_hashes.txt
```

Step 5 — crack with John

```
./john/run/john --format=bcrypt --wordlist=rockyou.txt bcrypt_hashes.txt
```

Show results:

```
./john/run/john --show --format=bcrypt bcrypt_hashes.txt
```



32 Passwords cracked, much slower than salted SHA 256

Week 7 Objective

The objective of Week 7 was to evaluate password strength by comparing weak and strong passwords using the previously implemented hashing mechanisms. This phase aimed to measure how password complexity affects resistance to password cracking attacks.

Methodology

To conduct the password strength evaluation, a dataset of 40 users was divided into two categories:

Weak Password Group

The weak password group included common passwords such as:

- 123456

- password
- letmein
- welcome
- admin
- batman
- freedom
- trustno1
- guest
- test123

These passwords were selected because they commonly appear in leaked password datasets such as RockYou.

Testing Procedure

The following steps were performed:

1. Weak and strong passwords were added to the SQLite test database
2. Passwords were hashed using:
 - SHA-256
 - Salted SHA-256
 - bcrypt
3. Dictionary attacks were performed using John the Ripper
4. RockYou wordlist was used for cracking attempts
5. Cracking time and success rate were recorded

Week 8 Objective

The objective of Week 8 was to analyze results from all testing phases and create comparison charts, tables, and final documentation.

Comparison Table

Method	Salted	Cost Factor	Cracked	Time	Security Level
--------	--------	-------------	---------	------	----------------

SHA-256	No	N/A	40/40	Seconds	Weak
Salted SHA-256	Yes	N/A	39/40	~2 sec	Moderate
bcrypt	Yes	12 (4096 iterations)	32/40	~4 minutes	Strong

Performance Comparison

SHA-256

- Fast hashing
 - Easy to crack
 - No built-in salt
 - Not recommended for password storage
-

Salted SHA-256

- Improved security
 - Prevents rainbow table attacks
 - Still vulnerable to dictionary attacks
-

bcrypt

- Built-in salt
- Adjustable cost factor
- Slow hashing
- Strongest method tested

Security Recommendations

Based on the results, the following recommendations were made:

1. Replace SHA-256 with bcrypt
2. Enforce strong password policies
3. Implement minimum password length
4. Require special characters
5. Use rate limiting on login attempts

Final Conclusion

This project demonstrated that password security depends on both hashing methods and password complexity. SHA-256 hashing alone provides insufficient protection against dictionary attacks. Adding salting improves security but does not eliminate vulnerabilities. bcrypt provides the strongest protection due to its adaptive hashing and built-in salting mechanisms.

The results confirm that strong passwords combined with bcrypt offer the highest level of protection against modern password cracking techniques.